

IEPS: Programming Assignment 2

Data Extraction and Embeddings

Dino Džaferagić, Tibor Koderman, Jerneja Krajcar

I. INTRODUCTION

This report describes the solutions for the second programming assignment (PA2). The assignment covers three main phases: 1) extracting and cleaning relevant information (boilerplate removal from HTML pages), 2) implementing a local vector database with text segments and 768-dimensional embeddings using PostgreSQL with the pgvector extension, and 3) building a retrieval program that returns the most relevant text segments for a Slovenian query, optionally improved by a cross-encoder reranker.

A. Website Filtering

Our PA1 corpus is restricted to MedOverNet (<https://medover.zurnal24.si/>), a Slovenian medical/wellness portal whose pages split into two structural classes: editorial articles and discussion forum threads. For PA2 we apply a two-stage filter:

- **URL gate.** Forum, category, search, login, and registration paths are filtered out at the URL level (substrings `/forum/`, `kategorija`, `iskanje`, `prijava`, `registracija`, `tag/`, `author/`). This is a fast, deterministic pre-cut.
- **Content gate.** A page that survives the URL filter must yield non-empty cleaned text; otherwise `cleaned_content` is stored as an empty string and no segments are generated.

The PA2 extraction dump contains 10,413 HTML pages. URL-class breakdown: 9,062 forum threads (`/forum/tema/`), 436 forum/category index pages, 128 tag/author/login/search-style pages, and **787 potential articles** outside all filtered prefixes. Of those 787, **779 yield non-empty `cleaned_content`** and are segmented and embedded; 8 fail the content gate (empty extraction result). The committed `pa1/db` artifact remains the smaller PA1 custom-format dump and is kept separate from the PA2 extraction database.

II. BOILERPLATE REMOVAL

To clean the stored HTML documents and remove formatting noise like menus, footers, and visual navigation elements, we used a hybrid approach:

- **XPath queries:** Searching for content within `<article>`, `<main>` tags, or elements functioning as content containers (e.g., `class` or `id` containing "content").
- **Heuristics for element removal:** Before extracting text, we intentionally stripped elements obviously containing noise using XPath (tags such as `<nav>`, `<footer>`, `<header>`, `<aside>`, `<script>`, `<style>`). We also checked attributes for hidden menus and toolbars.

The cleaned text is then stored in a newly created `cleaned_content` column in our relational database.

III. TEXT SEGMENTATION

Properly segmenting the cleaned text is the most critical step to avoid losing context during vectorization and to ensure accurate information retrieval, as emphasized in the task requirements. Without a proper approach, important indicators get overlooked, leading to incomplete or out-of-context vector matches.

A. Criteria for Splitting the Page into Thematic Sections

As required, we had to define strict criteria for splitting the page into multiple thematic sections. Rather than relying on unpredictable HTML tags like paragraphs or sentences—which can range from two words to two pages—we chose length-based slicing constraints as our primary criteria. Our strict segmenting criteria are defined by two boundaries: 1) the maximum allowed volume of content per chunk (character count vs. word count) and 2) the adherence to syntactic borders (ignoring spaces vs. keeping whole words). By comparing these extremes, we established two distinct profiles:

B. Short segments (up to 50 characters)

This strategy arbitrarily slices the plain text into rigidly fixed small windows of up to 50 characters, often cutting mid-word or mid-sentence.

- **Advantages:** Yields highly dense micro-data with low vector variance. Performs excellently when the model only needs to pinpoint a highly specific phrase, isolated keyword, or independent date, acting similarly to exact keyword-matching but within a semantic space.
- **Disadvantages (Context Loss):** Slicing mid-word can severely confuse the embedding model, as the cut-off thought or sentence loses syntactic structure. By ignoring sentence or paragraph boundaries, we completely destroy the broader logic and context of the author’s original message.

C. Logical & Hybrid Overlapping Segments (~250 words)

To achieve the most appropriate strategy for vector embedding, we replaced fixed slicing with *Semantic/Logical Chunking* paired with a sliding window (overlap). Rather than blindly counting up to 250 words, this strategy analyzes the raw `\n\n` paragraph breaks and `[.!?]` sentence boundaries to group complete thematic thoughts:

- **Advantages (High Context Preservation & Overlap):** This strategy adheres perfectly to the assignment’s goal of making each segment a "complete description of one topic." We group paragraphs dynamically until hitting the word limit. If a block is too large, it correctly falls back to sentence-level boundary splits. Crucially, we introduced an **overlap of 50 words** between consecutive chunks to ensure no topic edge gets sharply severed, resulting in industry-grade RAG precision.
- **Disadvantages:** The dynamic processing requires more computational overhead (regex splitting over sentences and overlap buffer tracking). Furthermore, dynamically bounded text arrays can slightly vary the semantic vector sizes compared to strictly fixed sizes, though standard transformers effectively pool this variance.

D. Structuring with Hybrid Metadata

Furthermore, the task prompted us to extract structured meaningful content (e.g., article titles, authors, and publication dates). While our segments rely purely on robust plain-text NLP boundaries, we actively align this with structural HTML metadata. During the extraction

phase, XPath was utilized to pull the article’s core attributes. These are formally injected into the database alongside the plain text—mapping the extracted title into the `heading` column and pushing dates and authors into the `metadata` JSON properties. This guarantees that while the text chunks themselves remain semantically raw for better embeddings, they are permanently anchored to their original structured metadata context.

IV. VECTOR EMBEDDINGS

A. Database Design and Schema Justification

While the assignment provided a minimal baseline schema (a single `page_segment` table), we intentionally expanded it to fully support the assignment’s encouragement to "experiment with different chunking strategies" and to include structural metadata. To cleanly separate, compare, and test the two contrasting segmentation strategies side-by-side, we created two dedicated tables: `page_segment_short` and `page_segment_long`.

To ensure a robust, production-ready environment, we introduced several critical columns beyond the basic requirements:

- `cleaned_content_hash`: Added to the main `page` table to serve as an idempotency key. It allows us to safely re-run the extraction scripts without duplicating the segmentation effort if the original HTML content has not changed.
- `segment_text`: Replaced the baseline column name (`page_segment`) to prevent confusing name collisions with the table itself, explicitly denoting the raw text payload.
- `segment_index`: Preserves the sequential order of segments, allowing the retrieval system to fetch neighboring chunks and reconstruct surrounding text flow if more context is needed.
- `page_type`: Categorizes the source document (e.g., article vs. forum), enabling efficient SQL pre-filtering to improve search precision before the semantic vector search even begins.
- `embedding` and `embedding_model`: Stores the actual 768-dimensional vector and explicitly tracks the model used. This allows safely generating and comparing multiple different embedding models inside the same document without overlapping constraints.
- `metadata`: A flexible JSONB column to dynamically store extracted structural data (like author names or `published_at` dates) mapped from XPath, without requiring rigid continuous SQL schema alterations.

- **created_at** and **embedded_at**: Essential auditing timestamps to track when segmentation and asynchronous vectorization occurred.

Indexing maps vectors using `vector_cosine_ops` on HNSW / IVFFLAT trees. The database also employs strong UNIQUE constraints (`page_id`, `segment_index`, `embedding_model`) to prevent duplicate embeddings upon repeated document processing.

B. Selected Model and Multilingualism

To properly process the Slovenian language, we employed the multilingual model `sentence-transformers/LaBSE`. In the implementation script (`compute_embeddings.py`), this model vectorizes the prepared fragments and saves them back into the before mentioned tables. The output of the *LaBSE* model is a numerical vector of size 768. The integration runs sequentially in batches of 100 documents to guarantee memory safety.

C. Regex-Based Extraction (Section 2.2)

As a co-equal extraction technique, we authored `extractor_regex.py`, a pure regular-expression extractor that operates directly on raw HTML bytes without any tree construction. It pulls:

- **Title**: First match among `<h1>`, Open Graph `og:title` meta tag, or `<title>` element.
- **Publication date**: `article:published_time` meta or ISO-8601 `datePublished` JSON-LD pattern.
- **Author**: `<meta name="author">` or JSON-LD `author.name`.
- **Body**: Strips all HTML tags (including inline script/style blocks), collapses whitespace, and returns the first 4000 characters.

Compared to XPath, regex extraction is faster (no DOM parse) but structurally fragile: it relies on attribute-value patterns that may break with CMS changes, fails silently on malformed nesting, and cannot navigate tree relationships. For MedOverNet’s consistent CMS, both methods produced equivalent metadata; for the body text, XPath’s structural understanding produced substantially cleaner output (fewer artefacts from navigation text and JavaScript), so XPath remains primary and regex is retained as the documented alternative.

V. SIMILARITY METRIC AND ANN INDEX

A. Choice of Similarity Metric

Three distance operators are available in pgvector:

- **Cosine distance** (`<=>`): measures angle between vectors; magnitude-invariant. Correct for sentence embeddings, whose useful information lives in direction, not length. LaBSE normalises output to unit sphere, so cosine and dot-product are equivalent, but cosine is semantically cleaner to reason about.
- **L2 (Euclidean) distance** (`<->`): sensitive to vector magnitude. Appropriate for models that do not normalise output; gives biased results with LaBSE because all vectors have equal norm.
- **Inner product** (`<#>`): identical to cosine on unit-norm vectors but stored as negative inner product in pgvector, requiring `ORDER BY ASC` which is confusing. No practical advantage here.

We chose cosine (`<=>`) as the default metric. The `demo.py -metric` flag allows ad-hoc switching to L2 or IP for comparison purposes.

B. ANN Index: IVFFlat vs. HNSW

pgvector supports two ANN index types:

- **IVFFlat**: partitions vectors into lists Voronoi cells at build time; queries probe probes cells. Faster to build; lower memory; recall degrades if probes is too small or data distribution shifts.
- **HNSW**: hierarchical navigable small-world graph; higher build cost and memory; typically higher recall at fixed query time; does not require a pre-build training pass.

We selected IVFFlat because our segment tables are populated in a single batch (not continuously updated), so the one-time build cost is acceptable and memory is constrained. The recommended `lists` value is $\approx \sqrt{N}$: for `page_segment_short` ($N \approx 51,394$) this gives ≈ 227 ; for `page_segment_long` ($N \approx 2,414$) this gives ≈ 49 . The migration used `lists=100` as a placeholder; after embeddings are populated and row counts confirmed, the indexes are recreated with corrected `lists` values.

VI. INFORMATION RETRIEVAL

A. Demo Program

The retrieval demo (`demo.py`) accepts a Slovenian query, encodes it with LaBSE, and performs an ANN cosine search against `crawldb.page_segment_long` (default) or `page_segment_short`. Results are printed with score, source URL, and a 200-character preview and saved to `eval/runs/<timestamp>_baseline.json`.

Table I shows the top-3 segments returned by the baseline system for each of the six evaluation queries.

Table I
TOP-3 RETRIEVED SEGMENTS PER QUERY (BASELINE,
COSINE, PAGE_SEGMENT_LONG, LABSE)

Query	Rank 1 (score)	Rank 2 (score)	Rank 3 (score)
<i>Kateri so simptomi visokega krvnega tlaka?</i>	0.451 — Blood clot / thrombosis article; partially relevant (cardiovascular)	0.458 — 6 tips for managing blood pressure	0.469 — Letter acronym with pressure management
<i>Kakšna prehrana je primerna pri sladkorni bolezni tipa 2?</i>	0.407 — Cardiometabolic factors; diabetes lifestyle (relevant)	0.444 — Myths about sugar and diabetes (relevant)	0.445 — Type 1 vs type 2 diabetes diet tips (relevant)
<i>Kako naravno premagati nespečnost in izboljšati spanec?</i>	0.465 — General healthy habits article (partially relevant)	0.479 — Unhealthy habits / sedentary lifestyle (off-topic)	0.505 — Evening snacks for better sleep (relevant)
<i>Boli me</i>	0.678 — Chronic pain in joints/back (generic but medical)	0.678 — Chronic pain in joints/back (generic but medical)	0.678 — Chronic pain in joints/back (generic but medical)
<i>Kakšne so pravne posledice neplačila prometne kazni?</i>	0.718 — Celiac disease insurance article (off-domain)	0.740 — Healthy lifestyle promotion (off-domain)	0.740 — Police warnings about online pharmacy fraud (off-domain)
<i>ATM</i>	0.720 — Gambling addiction article (unrelated)	0.776 — Hodgkin lymphoma patient story (unrelated)	0.780 — Hemophilia / Mont Blanc story (unrelated)

shifts to a police/pharmacy article (slightly more "legal-sounding") but all scores are near zero — honest failure.

- **"ATM" (acronym):** All rerank scores < 0.03 . The cross-encoder correctly identifies that none of the medical segments are relevant.

C. Remaining Reranker Failure

The legal query *"Kakšne so pravne posledice neplačila prometne kazni v Sloveniji?"* remains the clearest failure

return what the index contains; a cross-encoder reranker cannot compensate for zero relevant documents.

B. Cross-Encoder Reranker

To improve retrieval quality on the three poor queries, we implemented a cross-encoder reranking stage (pipeline/rerank_crossencoder.py) using the BAAI/bge-reranker-v2-m3 model. The cross-encoder jointly encodes the query and each candidate segment, producing a relevance score that is superior to the bi-encoder cosine score because it captures query-segment interaction directly.

The reranker is invoked with `-rerank`. It fetches $4 \times k$ candidates from the vector index and then applies the cross-encoder to select the final top- k .

- **Good queries (krvni tlak, sladkorna bolezen, nespečnost):** Reranker significantly improves precision. For the insomnia query, baseline rank 1 was a generic "healthy habits" article (cosine 0.465); after reranking, a dedicated melatonin/sleep article rose to rank 1 with rerank score 0.991. For the blood pressure query, baseline returned a blood-clot article at rank 1 (cosine 0.451); after reranking, the most relevant blood-pressure management article (cosine 0.458) was promoted to rank 1 (rerank 0.993).
- **"Boli me" (ambiguous pain):** Reranker reshuffled to more specific pain articles (chronic pain overview, rank 1 rerank 0.307). Scores are low overall — the query is too short for meaningful cross-encoder scoring. Marginal improvement.
- **"Pravne posledice neplačila kazni" (legal, out-of-domain):** All rerank scores collapsed to ≈ 0.0001 . The cross-encoder correctly identifies that none of the medical corpus segments are relevant. Top result

VII. LIMITATIONS

- **Artifact scope.** The PA2 extraction dump is intentionally separate from the PA1 submission dump. It contains the page table, lookup tables, article link graph, and segment tables with embeddings; raw crawl operational tables such as `frontier_queue`, `link`, `image`, and `page_data` are excluded from the PA2 artifact to keep the database focused on extraction and retrieval.
- **IVFFlat recall sensitivity.** With `probes=1` (default), IVFFlat can miss relevant segments in adjacent Voronoi cells. For production use, `probes` should be tuned to $\approx \sqrt{\text{lists}}$.
- **Short-segment quality.** The 50-character short-segment strategy frequently cuts mid-word and mid-sentence, producing fragments that confuse both LaBSE and the cross-encoder. Short segments yielded noticeably lower retrieval precision in our manual spot-checks.
- **Slovenian-only coverage.** MedOverNet content is Slovenian; the corpus contains no multilingual articles, so the multilingual capacity of LaBSE and `bge-reranker-v2-m3` is underutilised here.

VIII. METHODS TRIED BUT NOT USED

- **Regex-only extraction as primary:** We implemented `extractor_regex.py` as a co-equal technique per assignment requirements. In practice, it produced adequate title and metadata extraction

but inferior body text (navigation artefacts, raw JS) compared to XPath. XPath remains primary; regex is retained for the Section 2.2 requirement.

- **Alternative embedding model:** We evaluated `paraphrase-multilingual-MiniLM-L12-v2` (384-d, ≈ 120 MB). Its smaller dimensionality (384 vs 768) is incompatible with the `vector(768)` schema, preventing direct comparison within the same table. In a qualitative side-by-side on 20 queries, LaBSE retrievals were consistently more semantically accurate for Slovenian; MiniLM occasionally matched on surface n-grams rather than meaning.
- **HNSW index:** Considered for higher recall at equal query latency. Rejected for this batch-only use case because IVFFlat’s lower memory footprint fits within the Docker container’s constraints and HNSW’s recall advantage is not measurable without a labelled evaluation set.
- **Sentence-level segmentation only:** Evaluated a pure-sentence splitter (no paragraph grouping). It produced over 180,000 very short segments for the same corpus, dramatically increasing embedding compute time and reducing per-segment context, with no retrieval quality benefit over the 250-word hybrid strategy.

IX. CONCLUSION

We built a complete end-to-end RAG extraction pipeline for MedOverNet: HTML boilerplate removal via XPath, dual-strategy text segmentation (50-char and 250-word hybrid), LaBSE 768-d embeddings stored in pgvector with IVFFlat cosine indexes, a flexible retrieval demo, and a cross-encoder reranker. The system reliably handles Slovenian medical and wellness content. The three well-formed queries consistently return relevant segments; the three adversarial queries expose the expected failure modes (underspecified input, out-of-domain corpus, ambiguous acronyms) that no retrieval system can fully overcome without richer data.